

Hello!

I am Sayeed Unnisa

I am here because I love to give presentations on **Java Full Stack**

You can find me at sayeedunnisa.js@gmail.com

The background is a solid teal color with a pattern of thin, wavy, concentric lines that create a sense of depth and movement. In the upper right and lower left corners, there is a subtle pattern of small, light-colored crosses or plus signs arranged in a grid-like fashion.

SPRING BOOT

Spring Boot

- Spring Boot is a module that reduces the complexity of development and the time required for development
- It is built upon the Spring framework.
- Spring Boot is a Spring based Project, which contains common code/configuration (Reduces common code written in Spring F/w)
- Spring Boot mainly provide Auto-Configuration.
- Spring Boot has provided starters (Pre-Defined Jars for every concept)

Ex:

spring-boot-starter-jdbc

spring-boot-starter-data-jpa

spring-boot-starter-web

spring-boot-starter-security

spring-boot-starter-mail

- Spring Boot applications must connect with Parent Project.
- Our Project is known as Child Project.
- We can create Spring Boot application either using Maven or using Gradle.
 - Project/Folder system is created
 - Dependency Management(JARs)
 - Goals/task (Compile, Build, Deploy..etc)
 - If we use Maven Project there will be pom.xml
for Gradle build.gradle files exist.

- Spring Boot do not support XML Configuration.
- It supports Java and Annotation based Configuration.
- Spring boot has provided Embedded Webservers like Apache Tomcat(default), Eclipse Jetty, JBoss Undertow to run our application
- Spring boot has provided Embedded Database [H2, HyperSQL, Derby]
- These are also called as In-Memory Database

Spring Initializr

- url: <https://start.spring.io/>
- This Website provides support to create Spring Boot application.
- Even if we use Any IDE(Eclipse, STS, VSCode, IntelliJIdea) that will finally connected to Spring Initializr

It has five sections

1. Build Tool (Maven/Gradle)
2. Programming Language (Java)
3. Spring Boot Version (3.0 / 2.7.x)
4. Project Metadata

GroupId : OrgName (com.oracle)

ArtifactId: ProjectName

Version : 6.1.2

Package name: basePackage (All your classes must be under given package or its sub-package)

5. Dependencies (JARs/Starters) : select Required Starters

1. Starter class / Main class

- This is an entry class in execution.
- It creates Spring Container (new Container : `ApplicationContext(I)`)
- Implementation class : `AnnotationConfigApplicationContext(C)`
- It loads all classes using `BasePackage`.
- All the classes which are in `basePackage` or `subPackage` are scanned by Spring Container and create objects.
- Provide data, read properties file, inject data , link objects..etc

2. Input File (`application.properties`)

- This file is used to pass input to container objects.
- It holds data in Key=Val format
- Keys are pre-defined here, we can pass our own keys(custom keys).
- Internally every key is a variable
- A key name can have symbols like dot(.) dash(-) and underscore(_)

No other symbols allowed

3. Build Information File (pom.xml)

This file contains details like

- Parent Project details
 <parent> </parent>
- Our project details
- Properties(Java Version, Cloud Version,..etc)
- Dependencies (JARs)
- Build Plugins (Compile-> Build)

Runners :

- To execute any logic only one time, while starting application (i.e. after creating container).
- We need to define one class that should implement interface `CommandLineRunner()`
- We need to override abstract method `run()` and define our logic.
- Main class/Starter class, after creating container, executing all runners in order.
- We can provide our own execution order using annotation `@Order()`

@ConfigurationProperties:-

- It is used to load multiple key=val data into variables.
- Must provide one common prefix for key name.
- Same prefix must be matched while loading keys in class
- Key name and variable name must be matching.
- Must generate set/get methods.

@ConfigurationProperties with Collections

It supports loading Collections data from Properties file of type

List, Set, Array , Map and Properties

➤ For List/Set/Array Syntax is

`prefix.variable[index]= value`

➤ For Map/Properties syntax is

`prefix.variable.mapKey=mapValue`

@ConfigurationProperties with Collections

- It supports loading Collections data from Properties file of type List, Set, Array , Map and Properties
- For List/Set/Array Syntax is
`prefix.variable[index]= value`
- For Map/Properties syntax is
`prefix.variable.mapKey=mapValue`

Environments

A Machine/System where our application is running successfully (deployed)

Example Environments are:

- 1) Dev Environment= Developer Machine
- 2) QA Environment= Testing Team
- 3) Stage Environment= Developer testing
- 4) UAT Environment= User Acceptance Test (Client Testing)
- 5) Prod Environment= Production Environment (Actual servers given to End Customers)

Profiles:

- Profiles help to have different application configuration for different environments
- Maintain multiple Properties for multiple environments.
- Based on Environment selected/running, matching properties file will be loaded.

Name Rule : application-{profilename}.properties

Activate : --spring.profiles.active={profilename}

- Our application code remains same in almost all environments.
But properties may get varied.

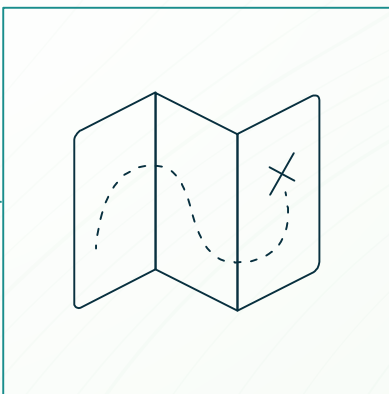
Ex: username and password of database

- Default Properties file will be loaded always.
- To activate any Profile, pass command line argument while running application.

Example :

file : application-qa.properties

arg : --spring.profiles.active=qa



Thanks!

Any questions?

You can find me at sayeedunnisa.js@gmail.com